Software Engineering Lab 7: Fuzz Testing

Finding and fixing input validation bugs with Hypothesis

Name: Abhigyan Dutta
SRN: PES2UG24CS019
Class: 5A, CSE
Date: 6 November 2026
Assigned repo: L6_L7_17_electricity_bill (Lab 7 folder)
My repo: github.com/UltraBot05/se-ps17-vaccination-scheduling, folder lab7-fuzz

1. The program

The handout calls the files processor.py and test_processor.py. In my repo they are `electricity_bill.py` and `test_electricity_bill.py`. The module has three functions that read text typed by the user:

- `units_consumed("1200:1350")` gives the units used between two meter readings
- `daily_average("5,7,6")` gives the average of the daily units
- `tariff_rate("DOM-A")` gives the rate per unit for a category and a slab

None of them checked their input.

2. Running the original program

Normal input works. But `500:100` (the meter going backwards) gives -400 units without any complaint, and `5,,6` crashes the whole program.

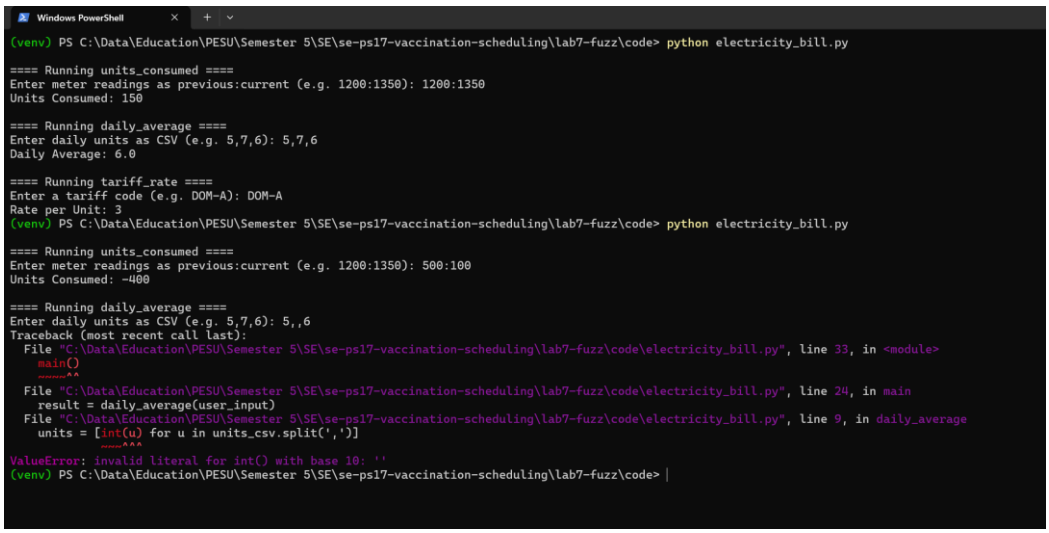


Figure 1: the original program, a normal run and then a run with bad input

3. Fuzz tests

Before writing the tests I had to decide what "not crashing" means. I chose this rule: a function either returns a correct answer or raises `ValueError` with a reason. Any other exception counts as a crash, and so does an answer that cannot be right, like negative units.

The test file has the three `_no_crash` tests from the starter, using `st.text() | st.none()`. I added a third strategy to each one that builds half valid strings out of numbers, blanks and junk, like `-5::12` or `DOM--b`, because purely random text rarely looks like a meter reading. Then I added four property

tests that build valid input and check the answer itself: units for two readings, the mean of a list, negative daily units being rejected, and every valid tariff code in any case with spaces around it.

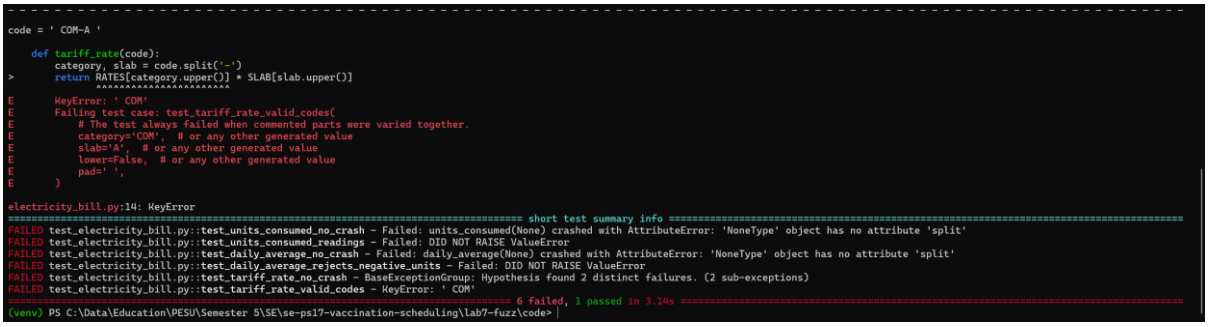


Figure 2: fuzz tests against the original code, 6 failed and 1 passed

Hypothesis shrank each failure to a small input:

Input	Original	Fixed
None	AttributeError in all three functions	ValueError: must be text
`500:100`	returns -400 units	ValueError: current reading is lower than previous
`5,,6`	crash on int('')	ValueError: must be a whole number
`-3 (daily)`	averaged as -3.0	ValueError: cannot be negative
XYZ-A	KeyError: 'XYZ'	ValueError: unknown category
` COM-A`	KeyError: ' COM'	accepted, rate 6
DOM-A-B	ValueError from unpacking, no clear reason	ValueError: must look like DOM-A

The one test that passed was `test_daily_average_matches_mean`. For clean input the arithmetic was already right, so it shows that the fix did not change any correct answers.

4. Fixing the code

- Each function first checks that it got text, then splits it and checks it has exactly the right number of parts.
- Every number goes through one helper that turns a bad `int()` into a clear ValueError and rejects negative values.
- units_consumed rejects a current reading lower than the previous one. daily_average rejects empty input and catches overflow for very large numbers. tariff_rate strips spaces, uppercases, and checks the code against RATES and SLAB before looking it up.
- `main()` catches ValueError and prints `Invalid input: <reason>` instead of a traceback.

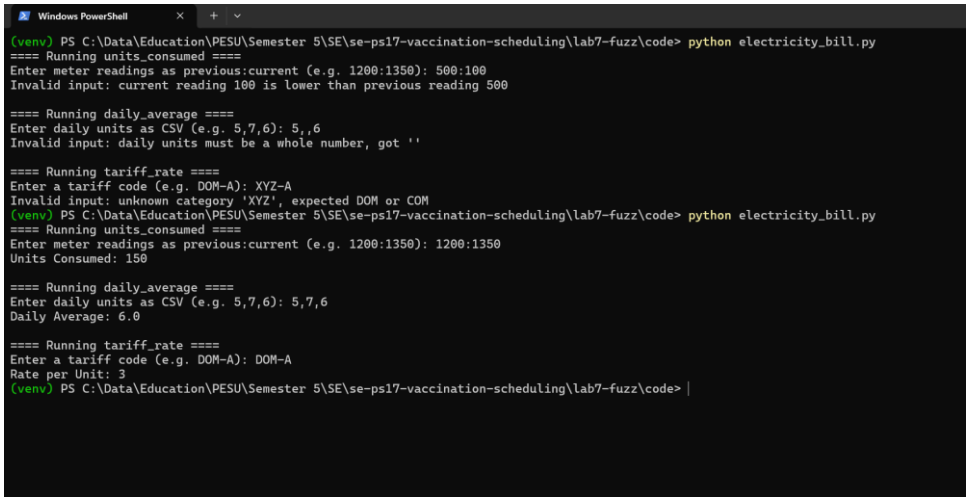


Figure 3: the fixed program, bad input first and then a normal run

5. Re-running the tests

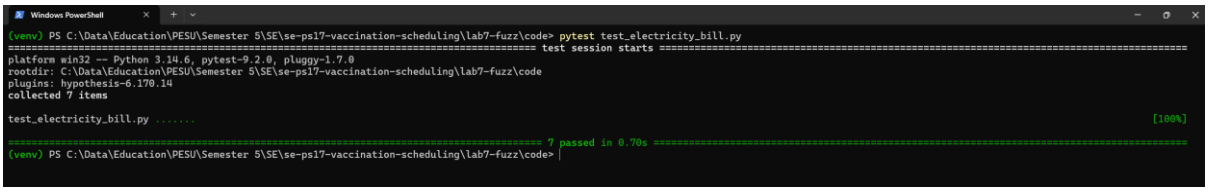


Figure 4: fuzz tests against the fixed code, 7 passed

By default Hypothesis tries 100 inputs per test. Before submitting I also ran the tests with 5000 inputs each, so 35,000 in total, and they still passed.

6. What I noticed

Most of the failures came from None and from a few characters in the wrong place, which a person typing tests by hand would probably skip. The KeyError for a code with a space around it is the kind of thing a real user would hit on day one.

Some decisions were mine and not the fuzzer's. A meter can roll over from 99999 back to 0, and my fix rejects that as a backwards reading, because the code has no idea of a maximum. Python's `int()` also accepts things like `1_000` and Arabic digits, so those still work. I left them in because they are real numbers, but a stricter version could allow only plain digits.